

CO 3 - ASSESSMENT TOOL -1

Name : Lanka Kavya
Registration Number : 192472298

Subject Code : CSA1003
Subject Name : Software Engineering

Hotel Reservation and Room Management Platform with Dynamic Pricing

1. Problem Analysis

The **Hotel Reservation and Room Management Platform with Dynamic Pricing** is a full-stack application designed to simplify hotel room booking and room management. The system allows customers to search for available rooms, view room details, make reservations, and receive dynamically calculated prices based on factors such as demand, occupancy, season, and availability.

Hotel administrators can manage rooms, room types, reservations, customers, pricing rules, and room availability through an administrative interface.

The platform uses **dynamic pricing** to automatically adjust room prices according to current booking demand and room availability. This helps hotels maximize revenue during high-demand periods while providing competitive prices during low-demand periods.

Objectives

- Provide an online hotel room reservation system.
- Allow customers to search and book available rooms.
- Enable administrators to add, update, and remove rooms.
- Maintain real-time room availability.
- Implement dynamic room pricing.
- Manage customer reservations efficiently.
- Prevent double booking of rooms.
- Provide centralized reservation and room management.
- Containerize the application using Docker.
- Use Git for version control and collaborative development.

Functional Requirements :

Customer Module

Room Management Module

Reservation Module

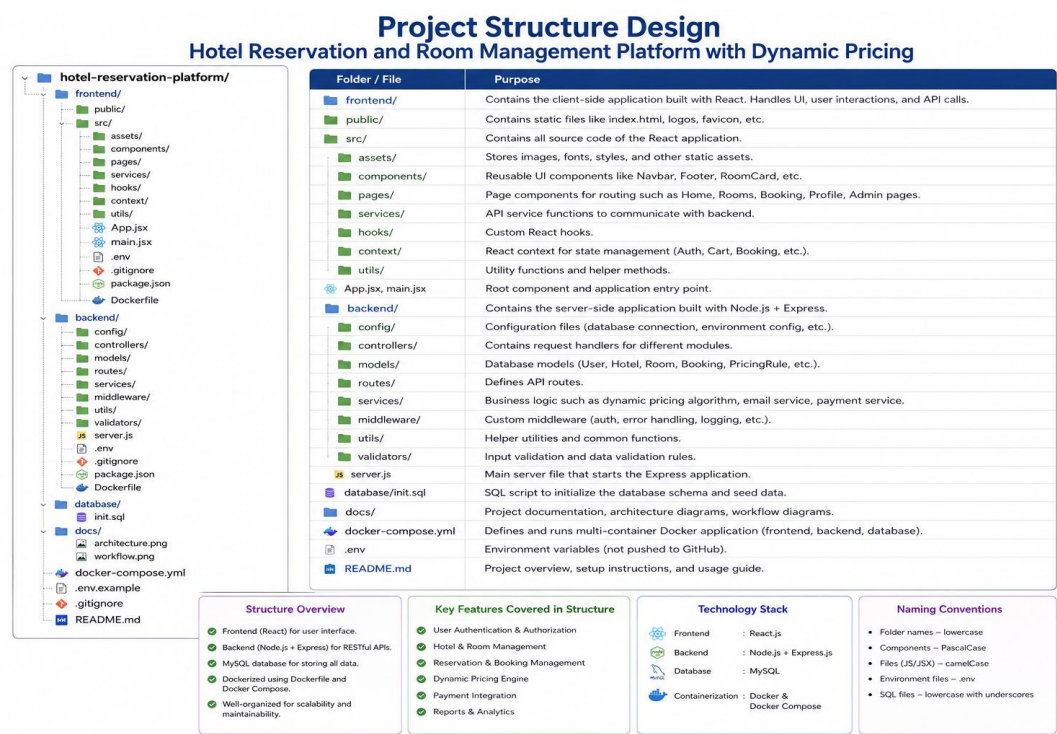
Dynamic Pricing Module

Admin Module

Expected Outcomes

- Faster hotel reservation processing.
- Accurate room availability.
- Reduced chances of double booking.
- Automated price calculation.
- Better hotel room utilization.
- Improved customer experience.
- Easier administration.
- Scalable deployment using Docker.

2. Project Structure Design



Folder Explanation

Folder/File	Purpose
frontend/	Contains the customer and admin user interface
backend/	Contains server-side application logic
controllers/	Handles application requests
models/	Defines database entities
routes/	Defines API endpoints
services/	Contains business logic such as dynamic pricing
database/	Contains database initialization scripts
docs/	Contains architecture diagrams and documentation
Dockerfile	Defines container configuration
docker-compose.yml	Runs multiple application services

Folder/File	Purpose
.gitignore	Prevents unnecessary files from being committed
README.md	Provides project documentation
.env	Stores configuration/environment variables

3. Git Repository Initialization

The project can be initialized using:

```
mkdir hotel-reservation-platform
cd hotel-reservation-platform
git init
```

Add project files:

```
git add .
```

Create the first commit:

```
git commit -m "Initial project setup"
```

Connect the local repository with GitHub:

```
git remote add origin <repository-url>
```

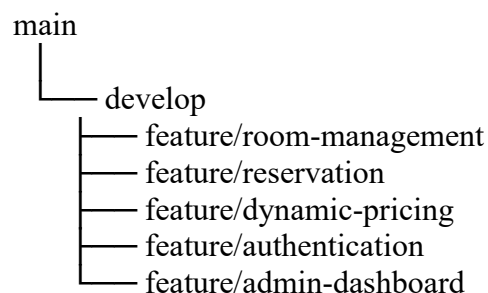
Push the project:

```
git branch -M main
git push -u origin main
```

Git is used because the assignment specifically requires repository initialization, meaningful commits, branching, merging, conflict handling, and synchronization.

4. Git Branching Strategy

A suitable branching model is:



Main Branch

Contains stable and production-ready code.

Develop Branch

Contains integrated development code before production release.

Feature Branches

Each major functionality is developed separately.

Example:

```
git checkout -b feature/dynamic-pricing
```

After completing the feature:

```
git add .  
git commit -m "Implement dynamic room pricing"
```

Then merge into development:

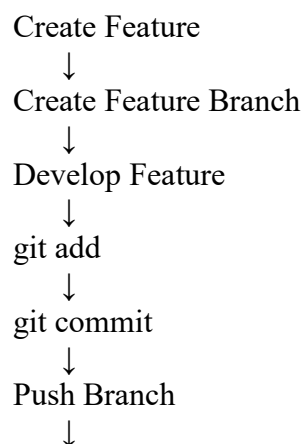
```
git checkout develop  
git merge feature/dynamic-pricing
```

Finally, the stable version can be merged into main.

This branching strategy is appropriate because the project contains multiple independent modules such as reservations, room management, authentication, and dynamic pricing.

5. Version Control Workflow

The workflow is:



Merge into Develop



Testing



Merge into Main

Example:

```
git checkout develop
git checkout -b feature/room-management
```

```
git add .
git commit -m "Add room management module"
```

```
git push origin feature/room-management
```

After testing:

```
git checkout develop
git merge feature/room-management
git push origin develop
```

For synchronization:

```
git pull origin develop
```

The assignment expects meaningful commits, branch creation, merging, synchronization, and conflict resolution where applicable.

6. Commit History

Example commit history:

```
Initial project setup
Add database schema
Implement user authentication
Add room management module
Implement reservation module
Add dynamic pricing algorithm
Add admin dashboard
Add Docker configuration
Add Docker Compose configuration
Fix reservation validation
Update project documentation
```

Good commit messages should be:

- Short.
- Clear.
- Meaningful.

- Related to one logical change.

Example:

```
git commit -m "Add dynamic room pricing"
```

instead of:

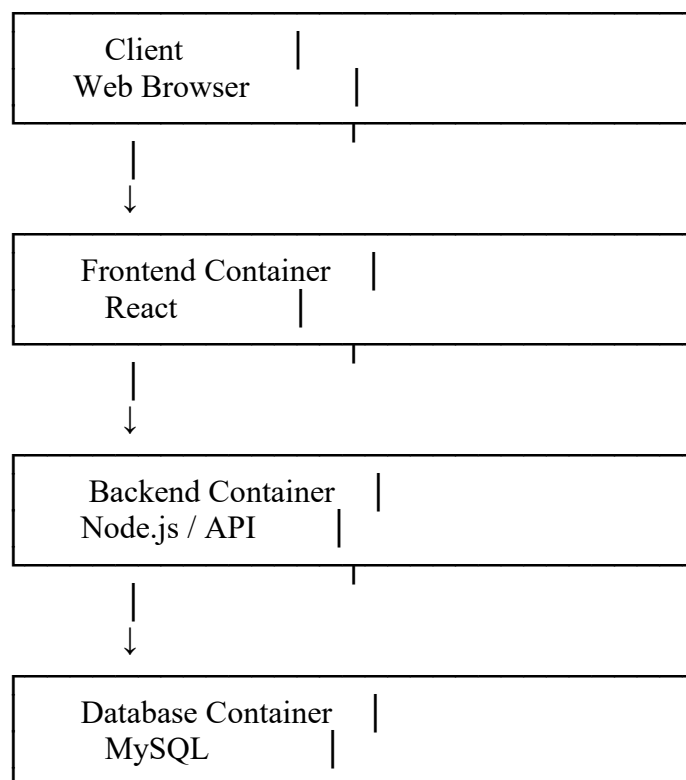
```
git commit -m "changes"
```

A clear commit history makes debugging, maintenance, rollback, and collaboration easier. The assignment explicitly evaluates meaningful commit history and professional commit messages.

7. Docker Environment Design

Docker is suitable because the hotel platform contains multiple components that can be deployed independently.

Required Containers



The assignment requires identifying the components that require containerization and explaining why Docker is appropriate.

8. Dockerfile Development

Backend Dockerfile

```
FROM node:20

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5000

CMD ["npm", "start"]
```

Explanation

Instruction	Purpose
FROM	Selects the Node.js base image
WORKDIR	Sets the application working directory
COPY	Copies project files
RUN npm install	Installs dependencies
EXPOSE	Documents the application port
CMD	Starts the backend server

The assignment requires the Dockerfile instructions and their purposes to be explained.

9. Docker Compose Design

Services

- **Frontend:** Provides the hotel booking interface.
- **Backend:** Provides APIs and business logic.
- **Database:** Stores rooms, users, bookings, pricing, and other information.

10. Container Deployment and Testing

Test the application by:

1. Opening the frontend.
2. Registering a customer.
3. Logging into the system.
4. Searching for rooms.
5. Checking room availability.
6. Making a reservation.

7. Checking the reservation details.
8. Verifying dynamic price calculation.
9. Logging into the admin dashboard.
10. Adding/updating room information.

11. Benefits of Git and Docker

Git Benefits : Collaboration, Version Management, Branching, Rollback, Conflict Management

Docker Benefits : Portability, Isolation, Scalability, Deployment, Maintainability